ACTIVIDAD 3

Programación orientada a objetos (POO)

Universidad Nacional De Colombia

Kevin David Aristizabal Giraldo

Docente: Walter Hugo Arboleda Mazo

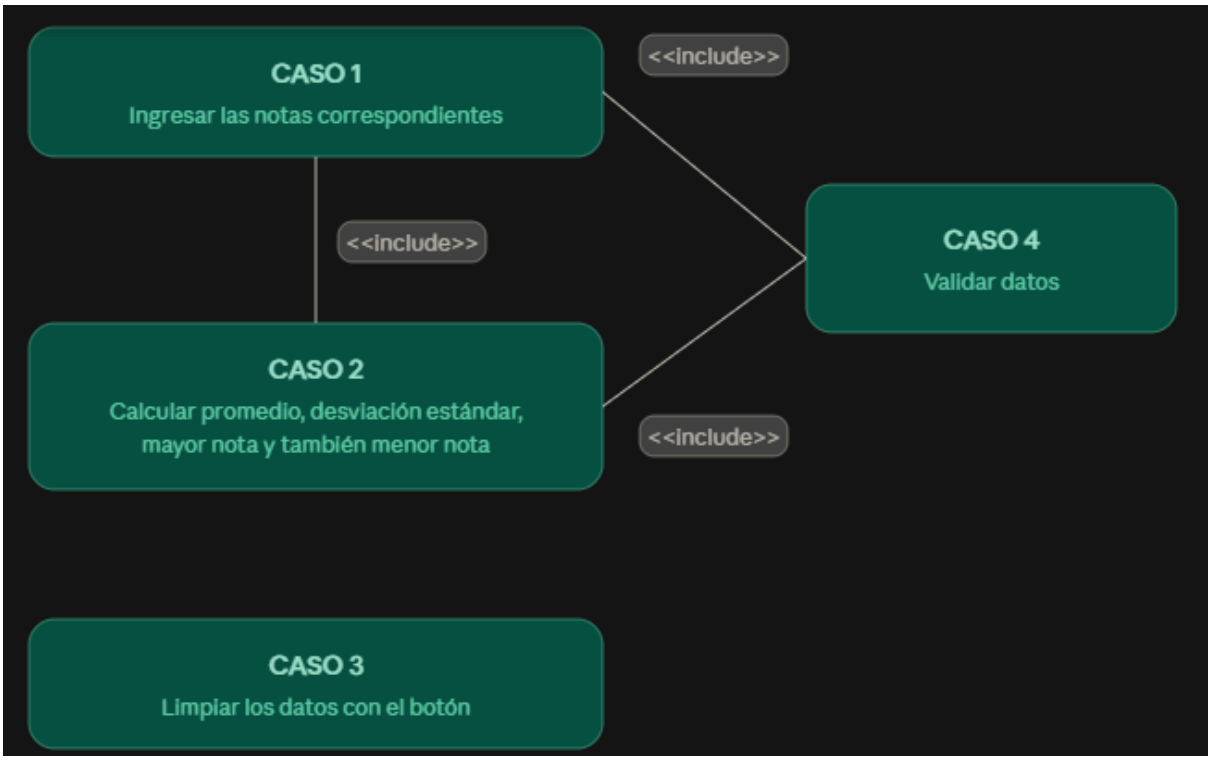
Octubre 2026

Ejercicio 8.2 Componentes Swing: NOTAS

Enunciado: Se pide programar una interfaz gráfica en la que se ingresen 5 notas de distintos estudiantes, y debe nombrar, la más alta, más baja, promedio y desviación estándar.

Parte propuesta: Se deben generar mensajes con el fin de que al ingresar determinados datos estos deban ser numéricos, además de ser numéricos se deben completar todas las casillas o si no aparecerá un mensaje de alerta para cada caso.

CASOS DE USO:



CASOS DE USO	REALIZADOR	DESCRIPCiÖN	FLUJOS ALTERNOS
CASO1: Ingresar las notas	Estudiante	Escribir cada nota en su campo de texto	Campo vacío: cuando la nota no se ingresa correctamente

CASO2: Calcular métricas	Sistema	Al pulsar calcular notas, lo que sucederá es que se te darán los datos de cual es la menor, cual es la mayor, promedio y desviación estándar	Si esto falla y no se calcula también se tiene una ventana de error
CASO3: Limpiar Datos	Sistema	Cuando pulsas limpiar se terminan limpiando los datos de todas las casillas	No tiene una ventana como tal cuando ocurre algo inesperado, ya que usualmente no debería suceder nada
CASO4: Validar datos	Sistema	Verificar que todas las notas existan y además de existir tengan un valor numérico propiamente dicho	Muestra un JOptionPane de error y de esta forma puedes saber cual campo falló

DIAGRAMA DE CLASES:

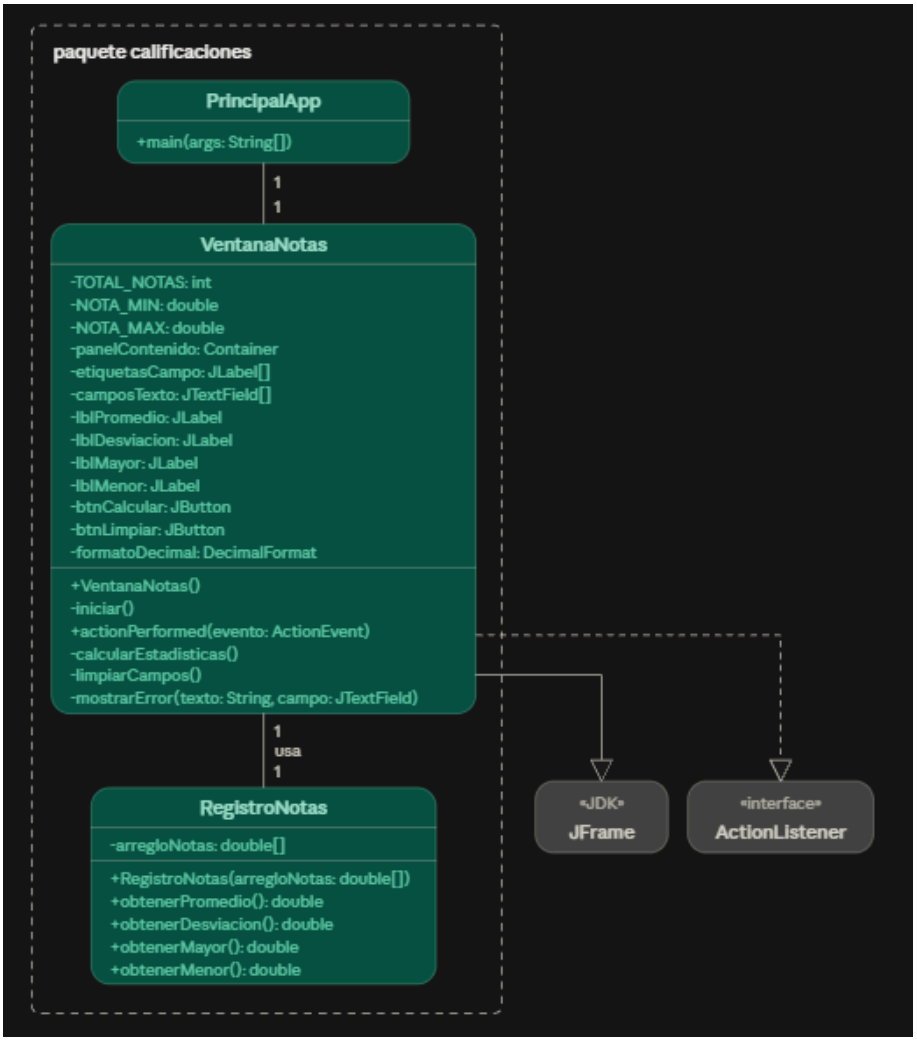
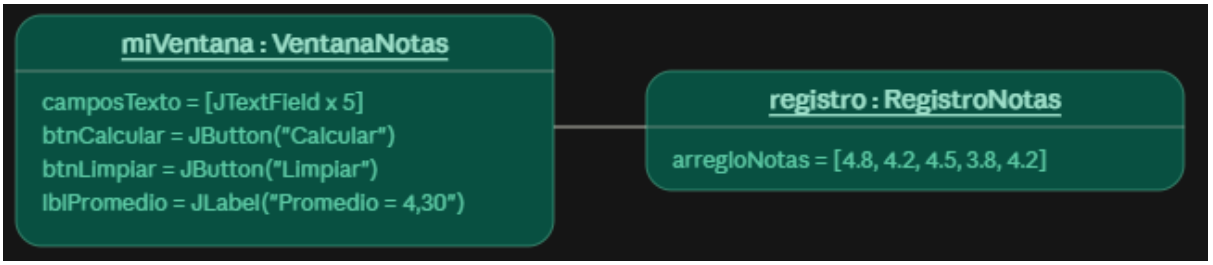
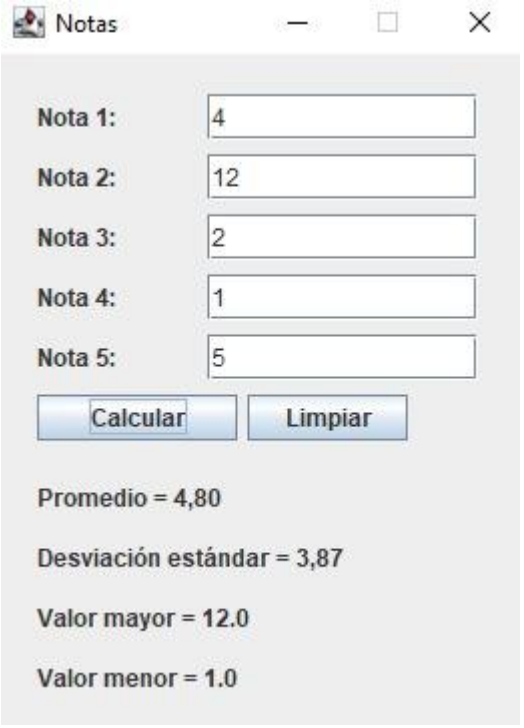


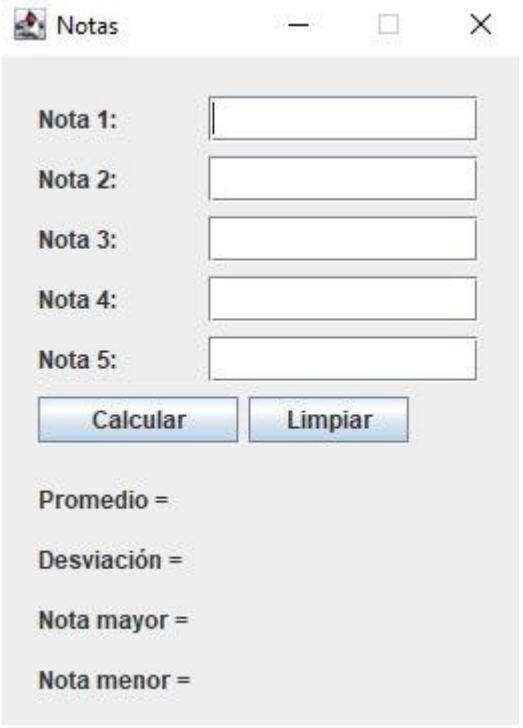
DIAGRAMA DE OBJETOS:

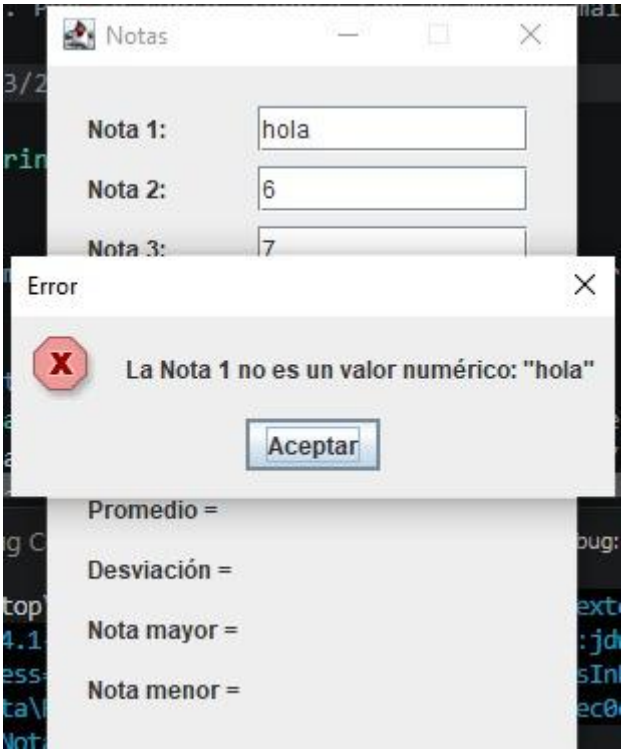


El punto de entrada es la clase Principal (método main), asociada uno a uno con VentanaPrincipal. Esta clase extiende JFrame e implementa ActionListener: contiene los componentes gráficos (etiquetas, campos de texto y botones) y gestiona los eventos en actionPerformed. VentanaPrincipal utiliza una instancia de Notas, clase de dominio que incluye un arreglo de notas y los métodos para calcular el promedio, la desviación estándar, el mayor y el menor. Así, la lógica de la interfaz queda separada.

INTERFAZ DE USUARIO:







CLASE: PRINCIPAL

```
package Notas;

public class Principal {


    public static void main(String[] args) {
        VentanaPrincipal miVentanaPrincipal; /* Define la ventana principal */
        miVentanaPrincipal = new VentanaPrincipal(); /* Crea la ventana principal */
        miVentanaPrincipal.setVisible(true); /* Establece la ventana como visible */
    }
}
```

CLASE: NOTAS

```
package Notas;


public class Notas {

    double[] listaNotas; /* Atributo que identifica un array de notas de tipo double */

    public Notas() {
        listaNotas = new double[5]; // Crea un array de 5 notas

    double calcularPromedio() {
        double suma = 0;
        for (int i = 0; i < listaNotas.length; i++) { // Se recorre todo el array
            suma = suma + listaNotas[i];              // Suma las notas del array
        }
              return suma / listaNotas.length;
    }


    double calcularDesviacion() {
        double prom = calcularPromedio(); /* Invoca el método para calcular el
promedio */
        double suma = 0;
        for (int i = 0; i < listaNotas.length; i++) {
            // Aplica fórmula para la sumatoria de elementos
            suma += Math.pow(listaNotas[i] - prom, 2);
```

```
        }
        return Math.sqrt(suma / listaNotas.length); /* Retorna el cálculo final */
    }


    double calcularMenor() {
        double menor = listaNotas[0]; /* Define una variable como la nota menor */
        for (int i = 0; i < listaNotas.length; i++) {
            if (listaNotas[i] < menor) {

                menor = listaNotas[i];
            }
        }
        return menor;
    }


    double calcularMayor() {
        double mayor = listaNotas[0]; /* Define una variable como la nota mayor */
        for (int i = 0; i < listaNotas.length; i++) {
            if (listaNotas[i] > mayor) {
                /* Si un elemento del array es mayor que el mayor actual,
                   se actualiza su valor */
                mayor = listaNotas[i];
            }
        }
        return mayor;
    }
}
```

CLASE: VENTANAPRINCIPAL

```
package Notas;

import java.awt.Container;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;
```

```
public class VentanaPrincipal extends JFrame implements ActionListener {

    // Un contenedor de elementos gráficos
    private Container contenedor;
    // Etiquetas estáticas de cada nota
    private JLabel[] etiquetasNota;
    // Etiquetas de resultados
    private JLabel promedio, desviacion, mayor, menor;
    // Campos de ingreso de cada nota
    private JTextField[] camposNota;
    // Botones para realizar cálculos y para borrar las notas
    private JButton calcular, limpiar;

    public VentanaPrincipal() {
        inicio();
        setTitle("Notas");              // Establece el título de la ventana
        setSize(280, 380);              // Establece el tamaño de la ventana
        setLocationRelativeTo(null);
        // Establece que el botón de cerrar permitirá salir de la aplicación
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
    }

    /**
     * Método que crea la ventana con sus diferentes componentes
     * gráficos
     */
    private void inicio() {
        contenedor = getContentPane();
        contenedor.setLayout(null);

        etiquetasNota = new JLabel[5];
        camposNota = new JTextField[5];

        // Etiqueta y campo de texto de cada nota
        for (int i = 0; i < 5; i++) {
            etiquetasNota[i] = new JLabel();
            etiquetasNota[i].setText("Nota " + (i + 1) + ":");
            etiquetasNota[i].setBounds(20, 20 + 30 * i, 80, 23);

            camposNota[i] = new JTextField();
            camposNota[i].setBounds(105, 20 + 30 * i, 135, 23);

            contenedor.add(etiquetasNota[i]);
```

```
        contenedor.add(camposNota[i]);
    }

    // Establece el botón Calcular
    calcular = new JButton();
    calcular.setText("Calcular");
    calcular.setBounds(20, 170, 100, 23);
    calcular.addActionListener(this);  /* Gestiona los eventos del botón */

    // Establece el botón Limpiar
    limpiar = new JButton();
    limpiar.setText("Limpiar");
    limpiar.setBounds(125, 170, 80, 23);
    limpiar.addActionListener(this);

    // Establece las etiquetas de resultados
    promedio = new JLabel();
    promedio.setText("Promedio = ");
    promedio.setBounds(20, 210, 200, 23);

    desviacion = new JLabel();
    desviacion.setText("Desviación = ");
    desviacion.setBounds(20, 240, 200, 23);

    mayor = new JLabel();
    mayor.setText("Nota mayor = ");
    mayor.setBounds(20, 270, 200, 23);

    menor = new JLabel();
    menor.setText("Nota menor = ");
    menor.setBounds(20, 300, 200, 23);

    // Se añade cada componente gráfico al contenedor de la ventana
    contenedor.add(calcular);
    contenedor.add(limpiar);
    contenedor.add(promedio);
    contenedor.add(desviacion);
    contenedor.add(mayor);
    contenedor.add(menor);
}


private boolean todosVacios() {
    for (int i = 0; i < 5; i++) {
```

```
        if (!camposNota[i].getText().trim().isEmpty()) {
            return false;
        }
    }
    return true;
}
private void mostrarError(String mensaje, JTextField campo) {
    JOptionPane.showMessageDialog(this, mensaje, "Error",
            JOptionPane.ERROR_MESSAGE);
    campo.requestFocus();
    campo.selectAll();
}

/**
 * Método que gestiona los eventos generados en la ventana principal
 */
@Override
public void actionPerformed(ActionEvent evento) {
    if (evento.getSource() == calcular) { /* Si se pulsa el botón Calcular */
        // 1. Si no se ingresó ninguna nota, se muestra un error general
        if (todosVacios()) {
            mostrarError("No se ingresó ninguna nota", camposNota[0]);
            return;
        }

        Notas notas = new Notas();        // Se crea un objeto Notas

        // 2. Se revisa cada nota: vacía o no numérica
        for (int i = 0; i < 5; i++) {
            String texto = camposNota[i].getText().trim();

            if (texto.isEmpty()) {
                mostrarError("La Nota " + (i + 1) + " está vacía", camposNota[i]);
                return;
            }

            // Se acepta coma o punto decimal; solo dígitos, signo y un separador
            String normalizado = texto.replace(',', '.');
            if (!normalizado.matches("[+-]?(\\d+(\\.\\d*)?|\\.\\d+)")) {
                mostrarError("La Nota " + (i + 1)
                        + " no es un valor numérico: \"" + texto + "\"",
                        camposNota[i]);
                return;
            }
```

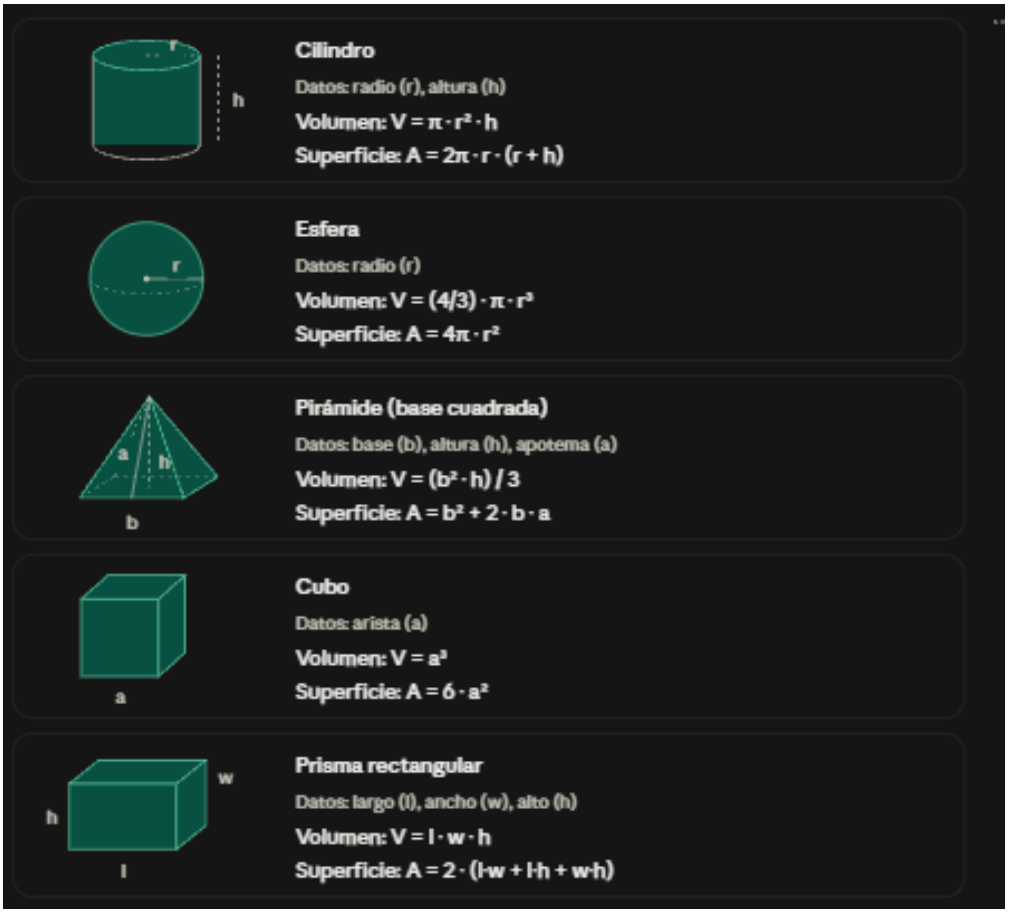
```
                notas.listaNotas[i] = Double.parseDouble(normalizado);
            }

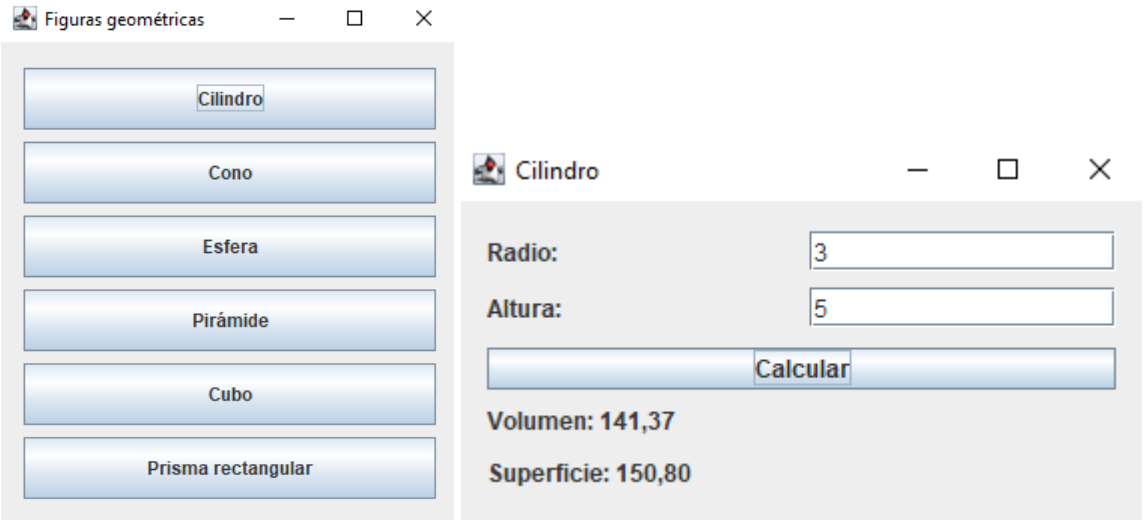
            // Se muestran los resultados formateados
            promedio.setText("Promedio = "
                    + String.format("%.2f", notas.calcularPromedio()));
            desviacion.setText("Desviación estándar = "
                    + String.format("%.2f", notas.calcularDesviacion()));
            mayor.setText("Valor mayor = " + notas.calcularMayor());
            menor.setText("Valor menor = " + notas.calcularMenor());
        }

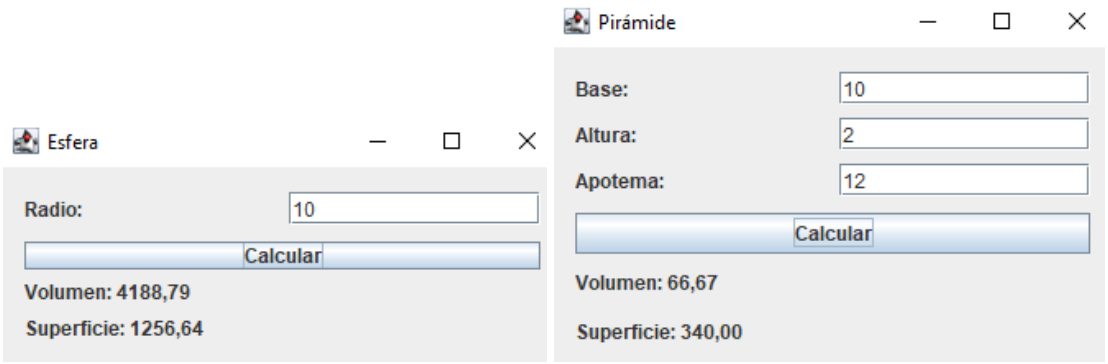
        /* Si se pulsa el botón Limpiar se dejan en blanco los campos de notas */
        if (evento.getSource() == limpiar) {
            for (int i = 0; i < 5; i++) {
                camposNota[i].setText("");
            }
            promedio.setText("Promedio = ");
            desviacion.setText("Desviación = ");
            mayor.setText("Nota mayor = ");
            menor.setText("Nota menor = ");
        }
    }
}
```

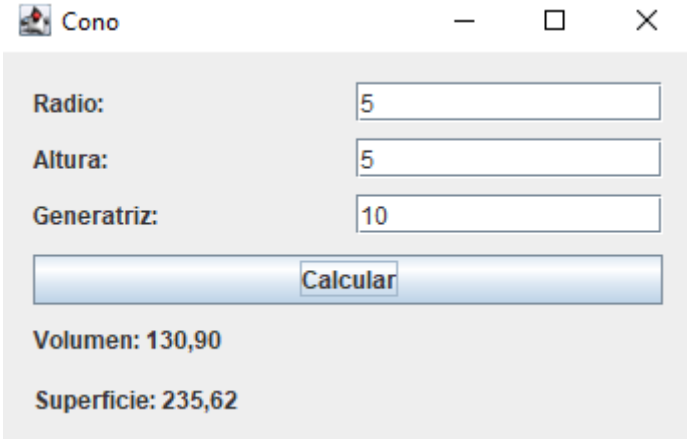
Ejercicio 8.3 Gestión de eventos

Enunciado: Programar una ventana que despliegue distintas figuras geométricas y sus variables puedan ser calculadas ingresando los valores pertinentes, siendo estas figuras (cilindro, esfera y la pirámide), dando los valores tanto de la región superficial de estas figuras como su región volumétrica.


r
h
Cilindro
Datos: radio (r), altura (h)
Volumen: V = π · r² · h
Superficie: A = 2π · r · (r + h)
r
Esfera
Datos: radio (r)
Volumen: V = (4/3) · π · r³
Superficie: A = 4π · r²
a
h
b
Pirámide (base cuadrada)
Datos: base (b), altura (h), apotema (a)
Volumen: V = (b² · h) / 3
Superficie: A = b² + 2 · b · a
a
Cubo
Datos: arista (a)
Volumen: V = a³
Superficie: A = 6 · a²
w
h
l
Prisma rectangular
Datos: largo (l), ancho (w), alto (h)
Volumen: V = l · w · h
Superficie: A = 2 · (lw + lh + wh)


Figuras geométricas
Cilindro
Cono
Esfera
Pirámide
Cubo
Prisma rectangular
Cilindro
Radio:
3
Altura:
5
Calcular
Volumen: 141,37
Superficie: 150,80





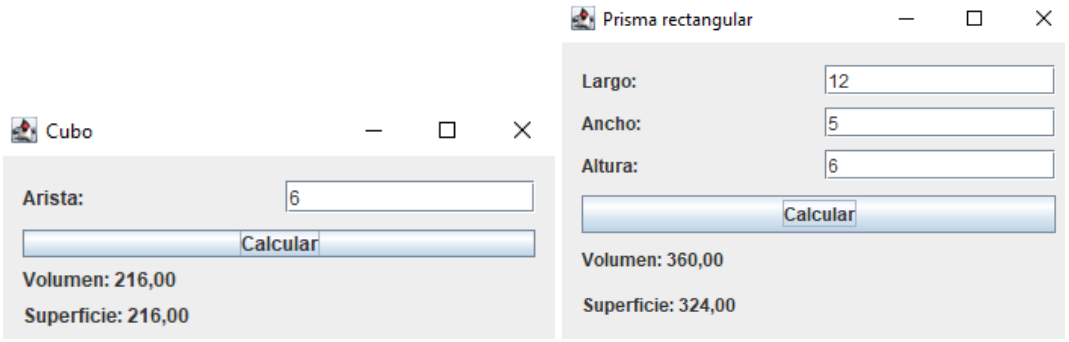
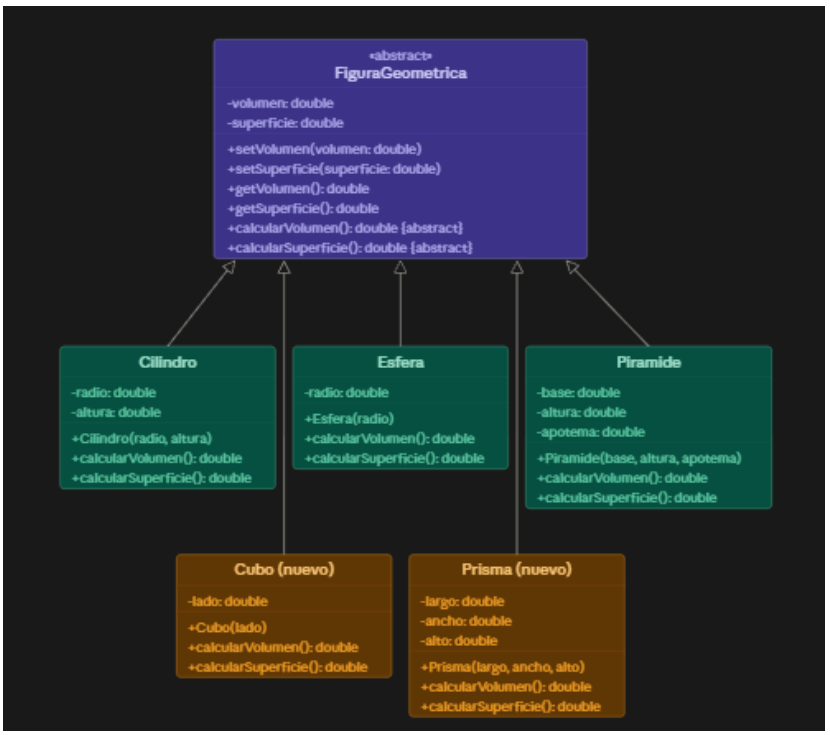
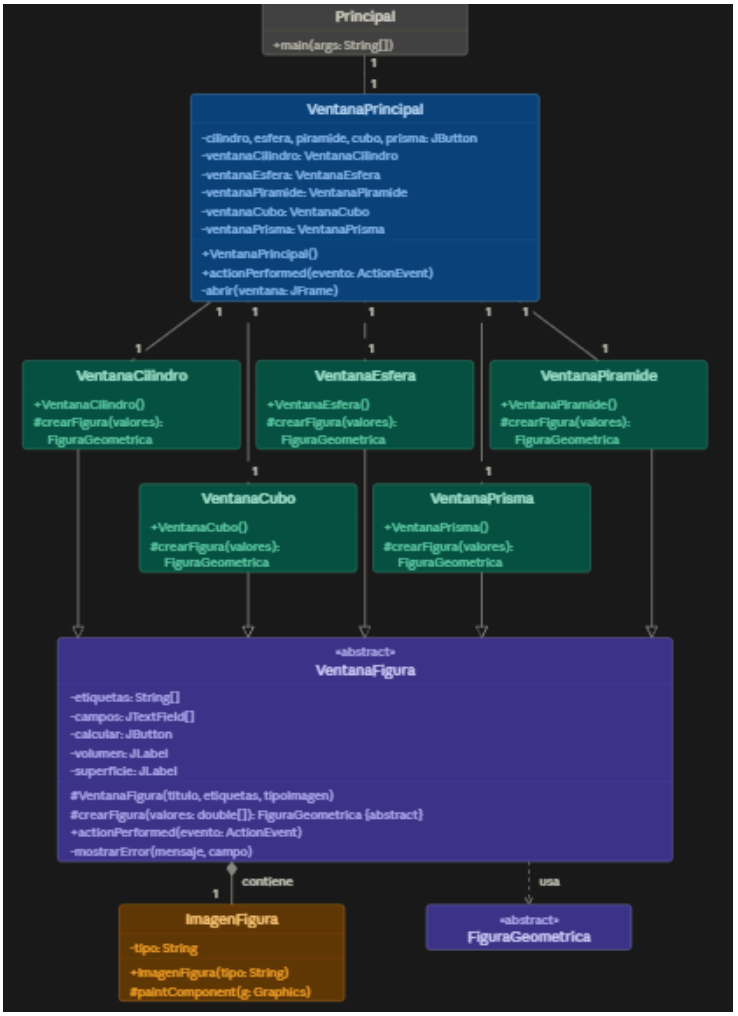


DIAGRAMA DE CLASES:


«abstract»
FiguraGeometrica
-volumen: double
-superficie: double
+setVolumen(volumen: double)
+setSuperficie(superficie: double)
+getVolumen(): double
+getSuperficie(): double
+calcularVolumen(): double {abstract}
+calcularSuperficie(): double {abstract}
Cilindro
-radio: double
-altura: double
+Cilindro(radio, altura)
+calcularVolumen(): double
+calcularSuperficie(): double
Esfera
-radio: double
+Esfera(radio)
+calcularVolumen(): double
+calcularSuperficie(): double
Piramide
-base: double
-altura: double
-apotema: double
+Piramide(base, altura, apotema)
+calcularVolumen(): double
+calcularSuperficie(): double
Cubo (nuevo)
-lado: double
+Cubo(lado)
+calcularVolumen(): double
+calcularSuperficie(): double
Prisma (nuevo)
-largo: double
-ancho: double
-alto: double
+Prisma(largo, ancho, alto)
+calcularVolumen(): double
+calcularSuperficie(): double


Principal
+main(args: String[])
VentanaPrincipal
-cilindro, esfera, piramide, cubo, prisma: JButton
-ventanaCilindro: VentanaCilindro
-ventanaEsfera: VentanaEsfera
-ventanaPiramide: VentanaPiramide
-ventanaCubo: VentanaCubo
-ventanaPrisma: VentanaPrisma
+VentanaPrincipal()
+actionPerformed(evento: ActionEvent)
-abrir(ventana: JFrame)
VentanaCilindro
+VentanaCilindro()
#crearFigura(valores): FiguraGeometrica
VentanaEsfera
+VentanaEsfera()
#crearFigura(valores): FiguraGeometrica
VentanaPiramide
+VentanaPiramide()
#crearFigura(valores): FiguraGeometrica
VentanaCubo
+VentanaCubo()
#crearFigura(valores): FiguraGeometrica
VentanaPrisma
+VentanaPrisma()
#crearFigura(valores): FiguraGeometrica
«abstract»
VentanaFigura
-etiquetas: String[]
-campos: JTextField[]
-calcular: JButton
-volumen: JLabel
-superficie: JLabel
#VentanaFigura(titulo, etiquetas, tipoImagen)
#crearFigura(valores: double[]): FiguraGeometrica {abstract}
+actionPerformed(evento: ActionEvent)
-mostrarError(mensaje, campo)
contiene
usa
ImagenFigura
-tipo: String
+ImagenFigura(tipo: String)
#paintComponent(g: Graphics)
«abstract»
FiguraGeometrica

CLASE CILINDRO:

```
package Figuras;

public class Cilindro extends FiguraGeometrica {

    public float radio;
    public float altura;

    public Cilindro(float radio, float altura) {
        this.radio = radio;
        this.altura = altura;
    }

    @Override
    public float calcular_volumen() {
        return (float) (Math.PI * radio * radio * altura);
    }

    @Override
    public float calcular_superficie() {
        return (float) (2 * Math.PI * radio * (radio + altura));
    }
}
```

CLASE CONO:

```
package Figuras;

public class Cono extends FiguraGeometrica {

    public float radio;
    public float altura;
    public float generatriz;

    public Cono(float radio, float altura, float generatriz) {
        this.radio = radio;
        this.altura = altura;
        this.generatriz = generatriz;
    }

    @Override
    public float calcular_volumen() {
        return (float) (Math.PI * radio * radio * altura / 3.0);
```

```
    }

    @Override
    public float calcular_superficie() {
        return (float) (Math.PI * radio * (radio + generatriz));
    }
}
```

CLASE CUBO:

```
package Figuras;

public class Cubo extends FiguraGeometrica {

    public float arista;

    public Cubo(float arista) {
        this.arista = arista;
    }

    @Override
    public float calcular_volumen() {
        return arista * arista * arista;
    }

    @Override
    public float calcular_superficie() {
        return 6 * arista * arista;
    }
}
```

CLASE ESFERA:

```
package Figuras;

public class Esfera extends FiguraGeometrica {

    public float radio;

    public Esfera(float radio) {
        this.radio = radio;
    }

    @Override
```

```
    public float calcular_volumen() {
        return (float) (4.0 / 3.0 * Math.PI * radio * radio * radio);
    }

    @Override
    public float calcular_superficie() {
        return (float) (4 * Math.PI * radio * radio);
    }
}
```

CLASE FIGURAGEOMETRICA:

```
package Figuras;

public abstract class FiguraGeometrica {

    protected float _volumen;
    protected float _superficie;

    public float volumen() {
        _volumen = calcular_volumen();
        return _volumen;
    }

    public float superficie() {
        _superficie = calcular_superficie();
        return _superficie;
    }

    public abstract float calcular_volumen();

    public abstract float calcular_superficie();
}
```

CLASE PIRÁMIDE:

```
package Figuras;

public class Piramide extends FiguraGeometrica {

    public float base;      // lado de la base cuadrada
    public float altura;
    public float apotema;
```

```
    public Piramide(float base, float altura, float apotema) {
        this.base = base;
        this.altura = altura;
        this.apotema = apotema;
    }

    @Override
    public float calcular_volumen() {
        return base * base * altura / 3;
    }

    @Override
    public float calcular_superficie() {
        return base * base + 2 * base * apotema;
    }
}
```

CLASE PRISMARECTANGULAR:

```
package Figuras;

public class PrismaRectangular extends FiguraGeometrica {

    public float largo;
    public float ancho;
    public float altura;

    public PrismaRectangular(float largo, float ancho, float altura) {
        this.largo = largo;
        this.ancho = ancho;
        this.altura = altura;
    }

    @Override
    public float calcular_volumen() {
        return largo * ancho * altura;
    }

    @Override
    public float calcular_superficie() {
        return 2 * (largo * ancho + largo * altura + ancho * altura);
    }
}
```

CLASE VENTANA PRINCIPAL:

```
package Figuras;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class VentanaPrincipal extends JFrame implements ActionListener {

   private final JButton btnCilindro = new JButton("Cilindro");
   private final JButton btnCono = new JButton("Cono");
   private final JButton btnEsfera = new JButton("Esfera");
   private final JButton btnPiramide = new JButton("Pirámide");
   private final JButton btnCubo = new JButton("Cubo");
   private final JButton btnPrisma = new JButton("Prisma rectangular");

   public VentanaPrincipal() {
      super("Figuras geométricas");
      setDefaultCloseOperation(EXIT_ON_CLOSE);
      setSize(320, 360);
      setLocationRelativeTo(null);

      JPanel panel = new JPanel(new GridLayout(6, 1, 8, 8));
      panel.setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

      JButton[] botones = {btnCilindro, btnCono, btnEsfera, btnPiramide,
btnCubo, btnPrisma};
      for (JButton b : botones) {
         b.addActionListener(this);
         panel.add(b);
      }
      add(panel);
   }

   @Override
   public void actionPerformed(ActionEvent e) {
      Object o = e.getSource();
      if (o == btnCilindro) {
         new VentanaCilindro().setVisible(true);
      } else if (o == btnCono) {
         new VentanaCono().setVisible(true);
      } else if (o == btnEsfera) {
```

```
            new VentanaEsfera().setVisible(true);
        } else if (o == btnPiramide) {
            new VentanaPiramide().setVisible(true);
        } else if (o == btnCubo) {
            new VentanaCubo().setVisible(true);
        } else if (o == btnPrisma) {
            new VentanaPrismaRectangular().setVisible(true);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new VentanaPrincipal().setVisible(true));
    }
}
```

ENLACE GITHUB: https://github.com/karistiqqq/POO-2026/tree/main